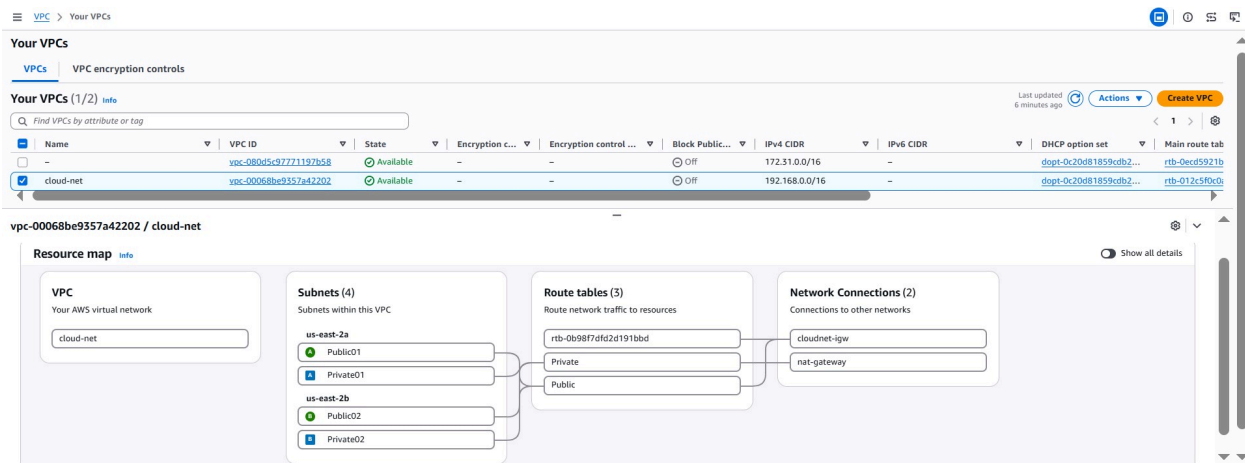


High Availability AWS Infrastructure Project

Step 1: Network Setup (VPC, Subnets, Route Tables, Security Groups)



Here's the resource map showing how everything in my network connects. These includes the VPC, both public and private subnets, the route tables, and the NAT Gateway all in one view.

For this project, I created a VPC called cloudnet to keep all resources in a single, isolated network.

Inside it, I set up four subnets:

- **public01 and public02** — resources that must be reachable from the internet (e.g., the jump server and load balancer)
- **private01 and private02** — the web servers, kept hidden from direct public access

I distributed the subnets across two Availability Zones so that if one zone has an issue, the application can continue running in the other.

I attached an Internet Gateway so the public subnets can reach the internet, and set up a NAT Gateway so the private servers can still download updates without being directly exposed.

For traffic control, I created three security groups:

- **public-sg** — allows SSH and HTTP access to the jump server
- **loadbalancer-sg** — allows HTTP/HTTPS traffic into the load balancer
- **private-sg** — allows traffic only from the load balancer and jump server (nothing directly from the internet)

The screenshot displays the AWS Management Console interface for VPCs and Subnets. The top section, 'Your VPCs (2)', shows a table with two VPCs: 'vpc-080d5c97771197b58' and 'vpc-00068be9357a42202', both in an 'Available' state. The bottom section, 'Subnets (4)', shows a table with four subnets: 'Private01', 'Public02', 'Private02', and 'Public01', all in an 'Available' state. The subnets are associated with the VPC 'vpc-00068be9357a42202'.

Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 CIDR
Private01	subnet-08dd936a1a1a8adf1	Available	vpc-00068be9357a42202 clou...	Off	192.168.3.0/24	-
Public02	subnet-02b1dd6d75ce23676	Available	vpc-00068be9357a42202 clou...	Off	192.168.2.0/24	-
Private02	subnet-03894f6c0aa495ad9	Available	vpc-00068be9357a42202 clou...	Off	192.168.4.0/24	-
Public01	subnet-0b406296002f7c717	Available	vpc-00068be9357a42202 clou...	Off	192.168.1.0/24	-

VPC overview and the 4 subnets.

☰ VPC > Route tables

Route tables (3) [Info](#) Last updated less than a minute ago [Actions](#) [Create route table](#)

Find route tables by attribute or tag

☐	Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC	Owner ID
☐	Private	rtb-0748a0d904ca49a97	2 subnets	–	No	vpc-00068be9357a42202 clou...	258233814516
☐	Public	rtb-012c5f0c0a5ae4a6c	2 subnets	–	Yes	vpc-00068be9357a42202 clou...	258233814516
☐	–	rtb-0ecd5921bea35e978	–	–	Yes	vpc-080d5c97771197b58	258233814516

Select a route table

☰ VPC > Internet gateways > igw-09c9c8c9a0916358b

Internet gateway igw-09c9c8c9a0916358b successfully attached to vpc-0eb5dfbee9146a7dc

igw-09c9c8c9a0916358b / cloud-net-igw

[Actions](#)

Details [Info](#)

Internet gateway ID igw-09c9c8c9a0916358b	State Attached	VPC ID vpc-0eb5dfbee9146a7dc cloud-net	Owner 258233814516
--	--------------------------------	---	---------------------------------------

Tags (1) [Manage tags](#)

Key	Value
Name	cloud-net-igw

☰ VPC > NAT gateways

NAT gateways (1) [Info](#) Last updated less than a minute ago [Actions](#) [Create NAT gateway](#)

Find NAT gateways by attribute or tag

☐	Name	NAT gateway ID	Connectivity...	State	State message	Availability ...	Route table ID	Primary public I...	Primary
☐	nat-gateway	nat-11f858f8f9e63e530	Public	Available	–	Regional	rtb-0b98f7dfd2...	3.21.42.39	–

Select a NAT gateway

Internet Gateway, NAT Gateway and route tables

Security group (sg-0adfa5bc62ba2a1d1 | loadbalancer-sg) was created successfully

Details

Security Groups (5) [Info](#)

Find security groups by attribute or tag

Name	Security group ID	Security group name	VPC ID	Description	Owner	Inbound rules count
Private-sg	sg-0ac539cd26f3f2454	private-sg	vpc-00068be9357a42202	for private	258233814516	1 Permission entry
-	sg-042e29b76a1132454	default	vpc-00068be9357a42202	default VPC security group	258233814516	1 Permission entry
Loadbalancer-sg	sg-0adfa5bc62ba2a1d1	loadbalancer-sg	vpc-00068be9357a42202	for the loadbalancer	258233814516	2 Permission entries
-	sg-07f1dc6f1a0205c5f4	default	vpc-080d5c97771197b58	default VPC security group	258233814516	1 Permission entry
Public-sg	sg-0d8ffba26694b1045	public-sg	vpc-00068be9357a42202	for public	258233814516	2 Permission entries

Select a security group

Security groups with their rules

Step 2: Amazon EFS

I set up Amazon EFS as shared storage so the web servers can access the same website files instead of keeping separate copies on each server.

I configured EFS across the VPC's two Availability Zones by creating a mount target in each zone. These mount targets provide network access to the EFS file system from the subnets where the application servers are running.

This lets the web servers in the private subnets share the same website files through EFS, supporting the high-availability design.

Amazon EFS > File systems

Elastic File System

File systems

Access points

AWS Backup [🔗](#)

AWS DataSync [🔗](#)

AWS Transfer [🔗](#)

Documentation [🔗](#)

File systems (1)

Filter by property values

Name	File system ID	Encrypt	Total size	Size in Standard	Size in IA	Size in Archive	Provisioned Throughput (MiB/s)	File system state	Creation time	Availability Zone	Replication overwrite protection
web-server-efs	fs-0bf73722132309c71	Encrypt	6.00 KiB	6.00 KiB	0 Bytes	0 Bytes	-	Available	Fri, 07 Aug 2026 15:21:41 GMT	Regional	Enabled

Access points (1)						
Search access points by name or ID						
	Name	Access point ID	File system ID	Path	POSIX user	Cr
☐	webserver-ap	fsap-0e1abc39ead51c576	fs-0bf73722132309c71	/	-	-

EFS file system overview

> fs-0c86372b47f8950c7

File system ID

fs-0c86372b47f8950c7

Lifecycle state

Available

Performance mode

General Purpose

Throughput mode

Bursting

Metered size

Monitoring

Tags

File system policy

Network

Mount targets (2)

Manage mount targets

A mount target enables an EC2 instance to connect to your file system. We recommend creating one mount target per availability zone.

Filter by property values

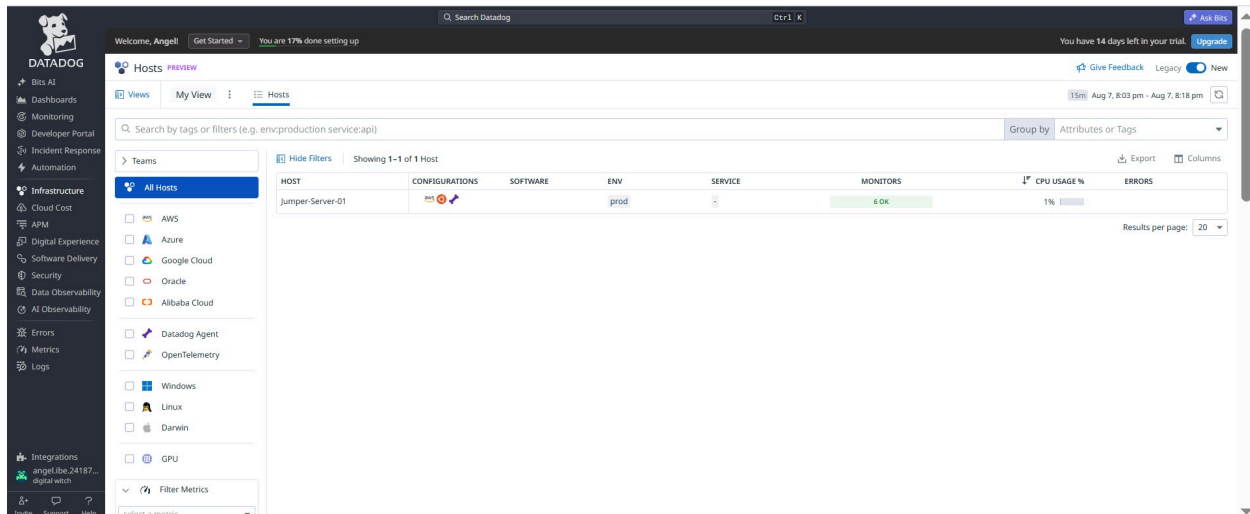
< 1 > ⚙

Availability Zone	Subnet	IP address	Security groups	Network interface ID	Mount target state
us-east-2a	subnet-0d1a7e2b1c3a45678 Private Subnet 01	192.168.1.67	sg-0a1b2c3d4e5f67890 Private-SG	eni-0a1b2c3d4e5f67890	Available
us-east-2b	subnet-0e2b3c4d5f6a78901 Private Subnet 02	192.168.2.43	sg-0a1b2c3d4e5f67890 Private-SG	eni-0b2c3d4e5f6a78901	Available

Mount targets in both Availability Zones

Step 3: Datadog Account

I created a Datadog account to monitor my servers (CPU usage, memory, and uptime) so I'd know if something went wrong instead of discovering it through a broken site.

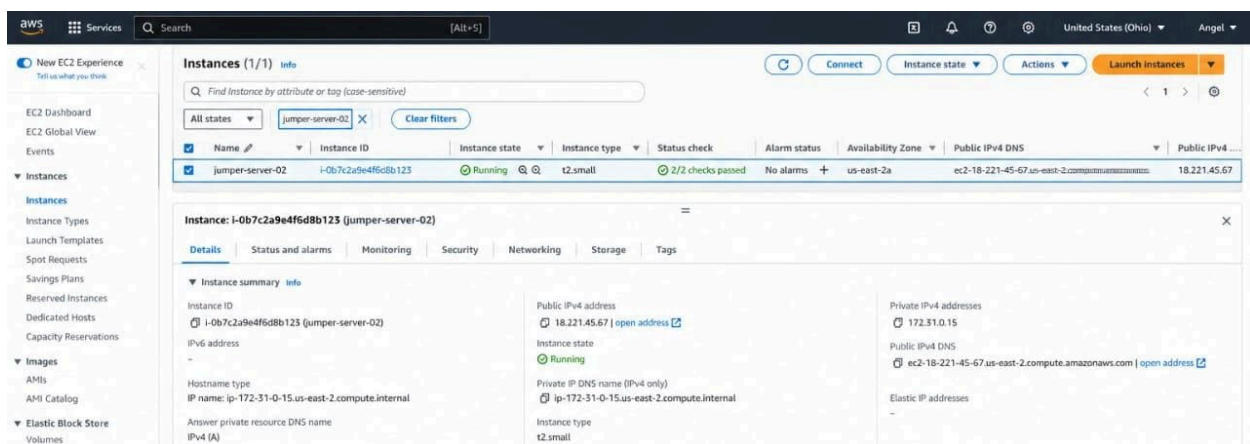


My Datadog account, showing it's set up and active under my name.

Step 4: Jump Server (Bastion Host)

Since my web servers sit in private subnets with no direct internet access, I set up a small jump server in the public subnet as my entry point.

I connect to the jump server first, then reach the private servers from there. This keeps the web servers hidden while still allowing access when needed.



Jump server running in the public subnet

```

ubuntu@jumper-server-02:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            967M   0 967M   0% /dev
tmpfs           195M  752K 194M   1% /run
/dev/xvda1      20G   5.1G 14G 27% /
tmpfs           973M   0 973M   0% /dev/shm
tmpfs           5.0M  4.0K 5.0M   1% /run/lock
tmpfs           973M   0 973M   0% /sysfs/cgroup
fs-0c06372b47f8950c7.efs.us-east-2.amazonaws.com:80E 6.0K 8.0E   1% /mnt/efs
ubuntu@jumper-server-02:~$ ls -la /mnt/efs
total 4
drwxr-xr-x 2 ubuntu ubuntu 6144 Aug 10 13:08 .
drwxr-xr-x 3 root    root   4096 Aug 10 12:50 ..
ubuntu@jumper-server-02:~$ sudo systemctl status datadog-agent --no-pager
● datadog-agent.service - Datadog Agent
   Loaded: loaded (/lib/systemd/system/datadog-agent.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2024-08-10 12:51:41 UTC; 23min ago
     Docs: https://docs.datadoghq.com/agent/
   Main PID: 1689 (agent)
     Tasks: 13 (limit: 1146)
    Memory: 7.442s
           CPU: /system.slice/datadog-agent.service
          CGroup: /system.slice/datadog-agent.service
                  └─1689 /opt/datadog-agent/ebc0ed0/bin/agent_run

Aug 10 12:51:40 jumper-server-02 systemd[1]: Starting Datadog Agent...
Aug 10 12:51:41 jumper-server-02 systemd[1]: Started Datadog Agent..
ubuntu@jumper-server-02:~$

```

```

Stopping Datadog Agent services...
Applying configurations...
Installing datadog-apm-inject...
Successfully installed datadog-apm-inject
Installing datadog-apm-library-java...
Successfully installed datadog-apm-library-java
Installing datadog-apm-library-python...
Successfully installed datadog-apm-library-python
Installing datadog-apm-library-ruby...
Successfully installed datadog-apm-library-ruby
Installing datadog-apm-library-js...
Successfully installed datadog-apm-library-js
Installing datadog-apm-library-dotnet...
Successfully installed datadog-apm-library-dotnet
Installing datadog-apm-library-php...
Successfully installed datadog-apm-library-php
Successfully ran the APM SSI install script in 17s!
/usr/bin/systemctl
* Starting the Datadog Agent...

Your Datadog Agent is running and functioning properly.
It will continue to run in the background and submit metrics to Datadog.
If you ever want to stop the Datadog Agent, run:

    sudo systemctl stop datadog-agent

And to run it again run:

    sudo systemctl start datadog-agent

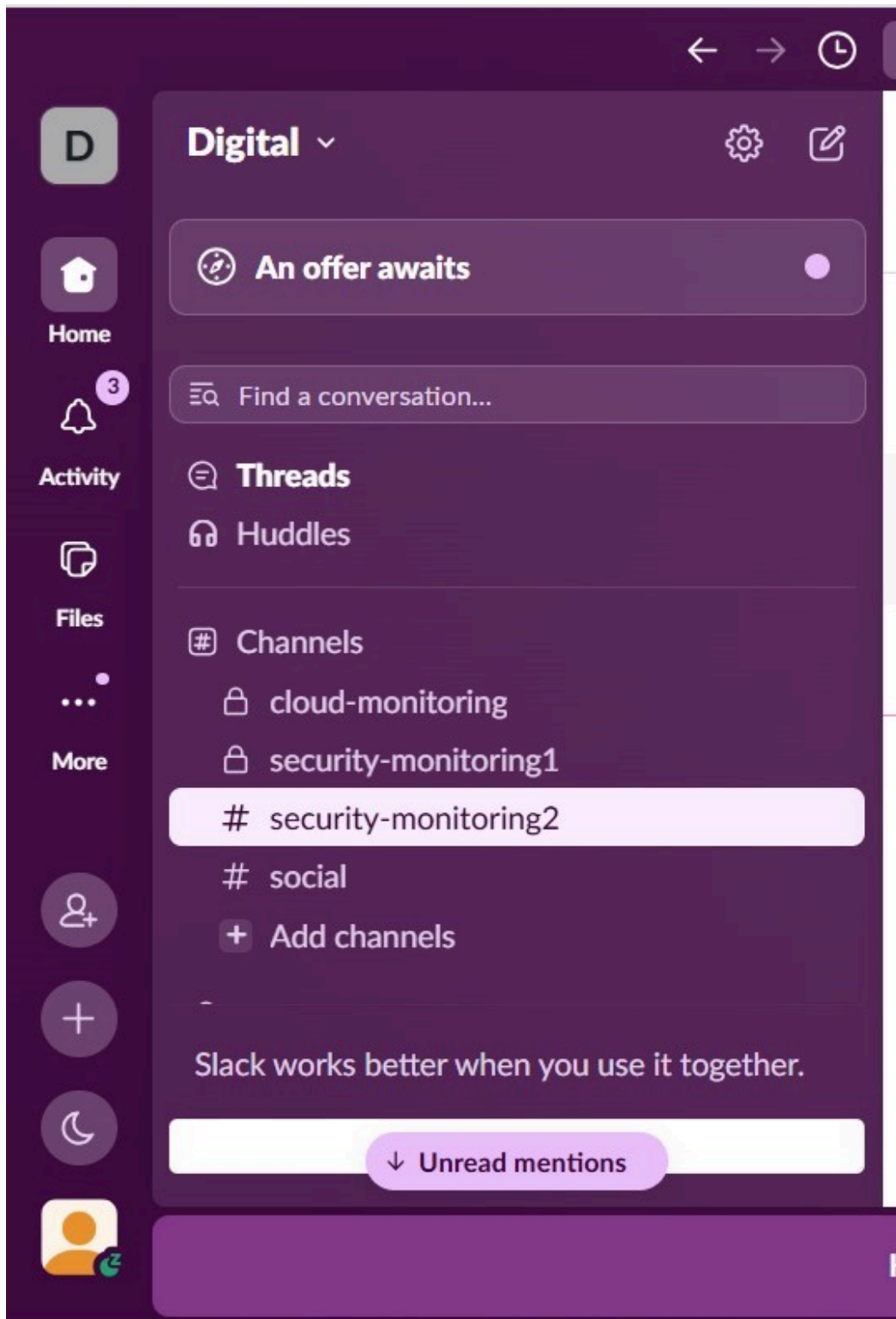
ubuntu@JumperServer-02:~$

```

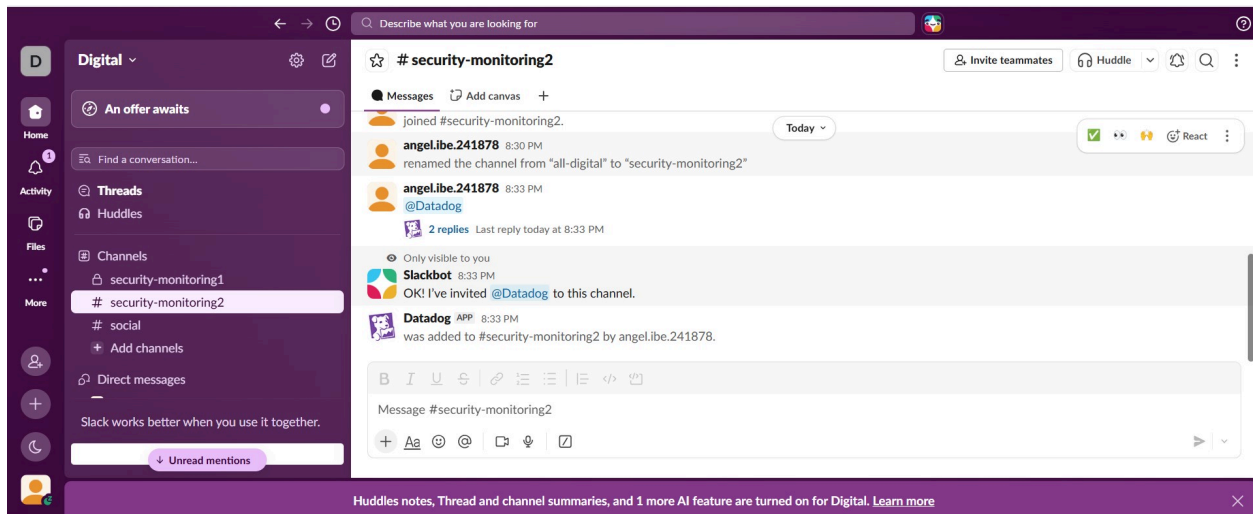
Terminal showing EFS mounted and Datadog agent running

Step 5: Slack Notifications

I created a Slack channel called security-monitoring2 and connected it to Datadog so alerts go straight to Slack instead of requiring manual dashboard checks.



Slack channel created for alerts

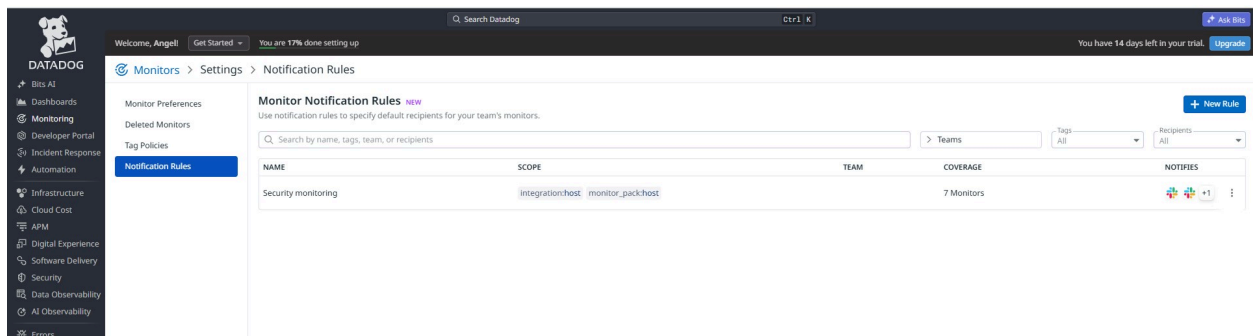


Datadog connected to Slack

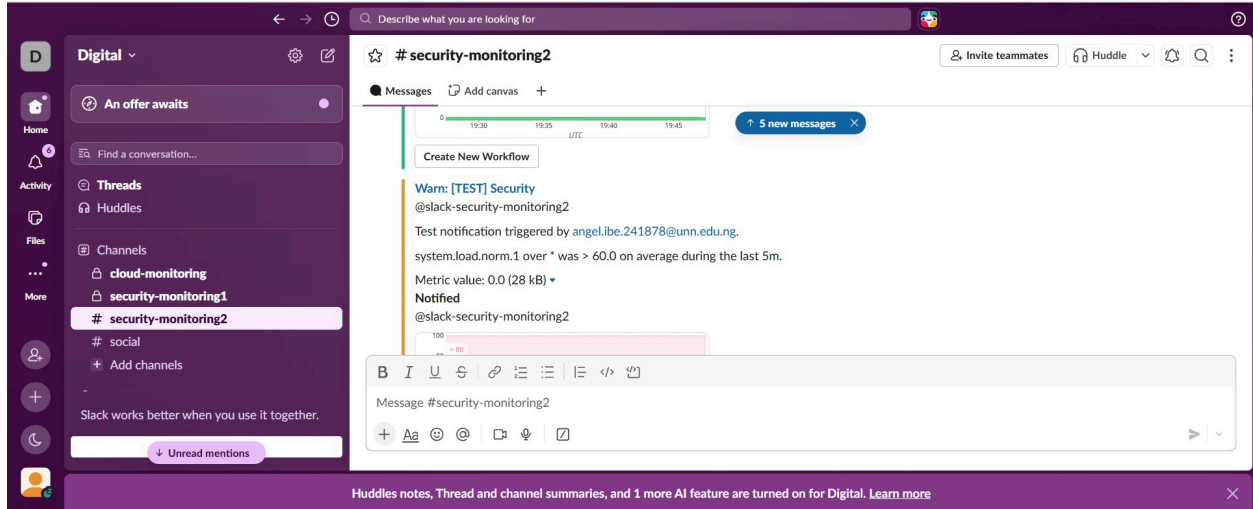
Step 6: Monitoring & Alerting

I set up a CPU monitor (warning at 60%, critical at 80%) and linked it to my Slack notification rule.

To test it, I intentionally pushed CPU usage higher and confirmed the alert appeared in Slack. This verified that the full pipeline works, not just that it's configured.



CPU monitor settings



Slack alert received during the test

Step 7: Deploying the Web Application

This is where everything comes together. I built a Launch Template that defines how each web server should be configured, then used it with an Auto Scaling Group so instances can be added or replaced automatically.

I created a Target Group to track instance health and placed a Load Balancer in front of everything to distribute traffic across the servers so no single instance carries all the load.

Launch templates (1) Info									
Search									
☐	Launch template ID	Launch template name	Default version	Latest version	Create time	Created by	Managed	Operator	
☐	lt-0b0ca528f73d47c01	mytemp	1	1	2026-08-07T14:30:24.000Z	arn:aws:iam::258233814516:root	false	-	

Launch Template settings

EC2 > Auto Scaling groups

Auto Scaling groups (1) Info Last updated less than a minute ago [Launch configurations](#) [Launch templates](#) [Actions](#) [Create Auto Scaling group](#)

Search your Auto Scaling groups

☐	Name	Status - new	Launch template/configuration	Instances	Instance health - new	Desired capacity	Min	Max
☐	webserver-auto	At desired capacity	mytemp Version Default	3	3/3 Healthy	3	3	12

Auto Scaling Group details

aws Services Search [Alt+S] US East (Ohio) cloudnet-demo

EC2 > Target groups > web-server-tg [Actions](#)

web-server-tg

Details

Target type Instance	Protocol : Port HTTP: 80	Protocol version HTTP1	ARN arn:aws:elasticloadbalancing:us-east-2:123456789012:targetgroup/web-server-tg/6d0f1e2a3b4c5d6e
IP address type IPv4	Load balancer web-server-lb	VPC vpc-0c1a2b3d4e5f67890	Target group type Application

Targets Monitoring Health checks Attributes Tags

Registered targets (3) [Deregister](#) [Register targets](#)

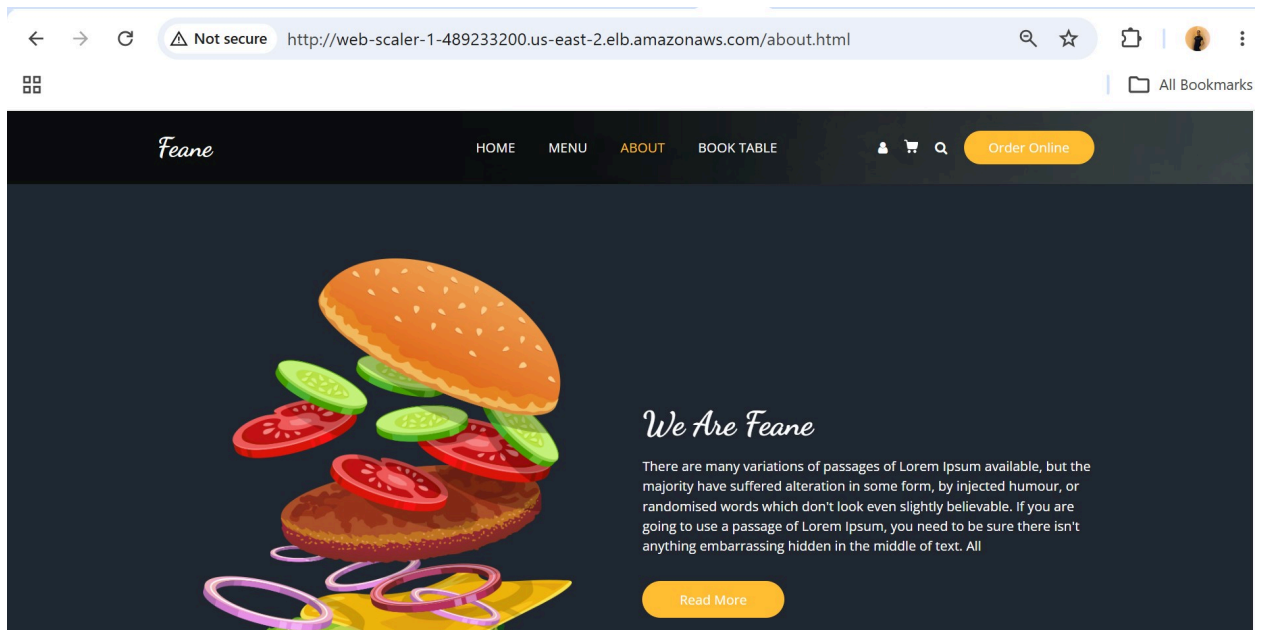
Filter targets

☐	Instance ID	Name	Port	Zone	Health status	Health status details	Last health check	Administrative state	Actions
☐	i-0a1b2c3d4e5f67890	web-server-01	80	us-east-2a	Healthy	-	30 seconds ago	InService	⋮
☐	i-0b1c2d3e4f5a67891	web-server-02	80	us-east-2b	Healthy	-	30 seconds ago	InService	⋮
☐	i-0c1d2e3f4a5b67892	web-server-03	80	us-east-2a	Healthy	-	30 seconds ago	InService	⋮

0 selected

✓ All targets are healthy.

Target Group showing all servers healthy



Site loading through the load balancer's link

Step 8: Domain Setup (Route 53)

I created a hosted zone in Route 53 and pointed my domain's nameservers to it so AWS manages DNS.

Then I added an A record that points the domain directly to my load balancer.

A screenshot of the AWS Route 53 console. The title is 'Hosted zones (3)'. Below the title is a search bar and a link to 'To change modes go to settings.'. On the right side of the console are buttons for 'View details', 'Edit', 'Delete', and 'Create hosted zone'. The main content is a table with the following columns: 'Hosted zone name', 'Type', 'Created by', 'Record count', and 'Description'. There is one row in the table with the following data: 'angelibedemo.click' (with a link icon), 'Public', 'Route 53', '5', and '-'.

Hosted zone name	Type	Created by	Record count	Description
angelibedemo.click	Public	Route 53	5	-

Hosted zone with nameservers

Public **angelibedemo.click** info Delete zone Test record Configure query logging 0 records selected Select a record to see its details

Hosted zone details Edit hosted zone

Records (5) Accelerated recovery DNSSEC signing Hosted zone tags (0)

Automatic mode is the current search behavior optimized for best filter results. [To change modes go to settings.](#)

Filter records by property or value Type Routing p... Alias < 1 > ⚙

☐	Record name	Type	Routin...	Differ...	Alias	Value/Route traffic to	TTL (s...	Health ...	Evalu...
☐	angelibedemo.click	A	Simple	-	Yes	dualstack.web-scaler-1-4892...	-	-	Yes
☐	angelibedemo.click	NS	Simple	-	No	ns-1918.awsdns-47.co.uk. ns-1043.awsdns-02.org. ns-550.awsdns-04.net. ns-370.awsdns-46.com.	172,800	-	-
☐	angelibedemo.click	SOA	Simple	-	No	ns-1918.awsdns-47.co.uk. a...	900	-	-
☐	_c21fe17dc448a98966a9c538c299295.angelibedemo.click	CNAME	Simple	-	No	_a203e71d6f36af10c782030...	300	-	-
☐	www.angelibedemo.click	A	Simple	-	Yes	dualstack.web-scaler-1-4892...	-	-	Yes

A record pointing to the load balancer

Step 9: HTTPS (SSL Certificate)

I requested a free SSL certificate through AWS Certificate Manager and validated it through Route 53. Once it was issued, I added an HTTPS listener on my load balancer and attached the certificate.

Now the site loads securely with a padlock instead of an unsecured warning.

AWS Certificate Manager > Certificates ⓘ 🔔 ⚙

Successfully deleted certificate ×
You have successfully deleted 1 certificate from your account in AWS Certificate Manager.

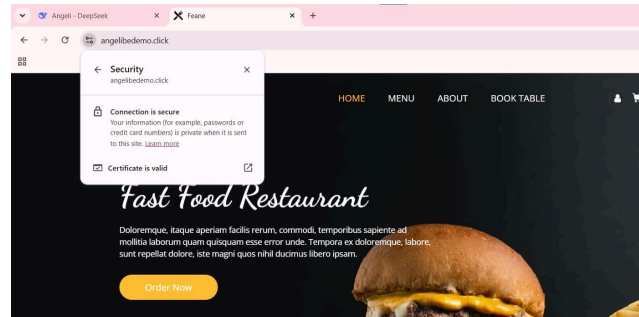
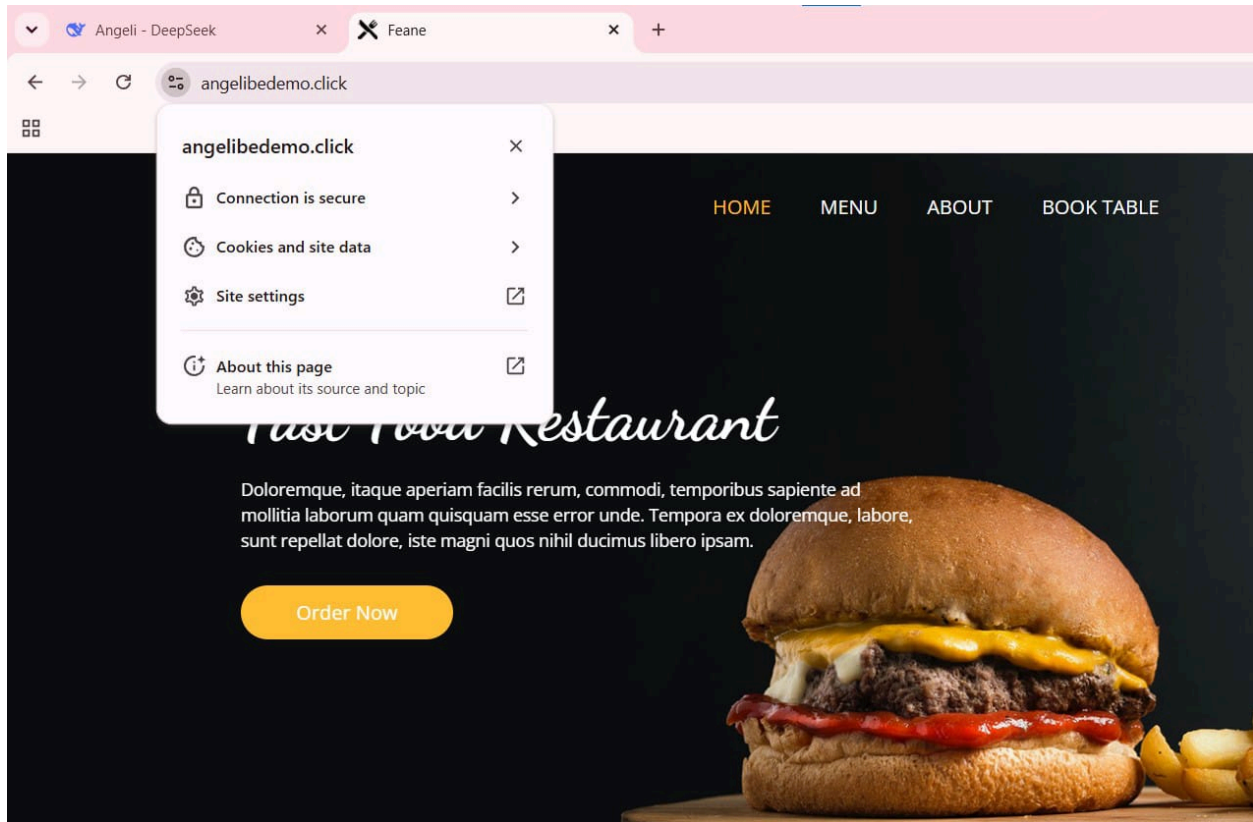
Update on ACM public certificate validity ×
We have updated the validity period for ACM public certificates to comply with new CA/Browser Forum guidelines. We have also reduced exportable certificate pricing. No action is required. [Learn more](#)

Certificates (1) ⚙ More actions Manage expiry events Import Request

Search certificates < 1 > ⚙

☐	Certificate ID	Status	Common name	Type	In use	Renewal eligibility	Key algorithm
☐	d63b3b53-4fe6-4585-b261-93309fb53ba0	Issued	angelibedemo.click	Amazon Issued	No	Ineligible	RSA 2048

Certificate showing Issued



Website successfully loading over HTTPS with Chrome confirming a secure connection.

That covers the full build; a load-balanced, auto-scaling setup with shared storage, real-time monitoring, Slack alerting, and a secured custom domain running over HTTPS. The biggest challenges were getting the load balancer's health checks to pass (I had a missing IAM role and the wrong security group attached to my web servers) and getting HTTPS to work correctly across both the www and non-www versions of my domain. Working through both gave me a

clearer understanding of how the pieces actually connect, not just how to follow the steps. The end result is a website that stays online even if a server fails, scales automatically with traffic, and alerts me the moment something needs attention.
